

Overclocking with the on-chip PLL

Driving the SoC clock from the iCE40 SB_PLL40_PAD

The iCEBreaker board only has a 12MHz crystal, but the picosoc runs faster than that by routing the crystal through the iCE40 SB_PLL40_PAD hard PLL before it reaches the SoC. The committed competition build runs at 27.75 MHz. The clock tree lives in `picosoc/icebreaker.v`.

How it works

The PLL multiplies the 12MHz input up to the target SoC clock, and that output is named `clk`. Every downstream block (reset logic, 7-segment driver, GPIO, `picosoc`) keeps using `clk` unchanged, so the PLL is inserted transparently between the pad and the core. The raw pin must stay a separate net, `clk_in`, because SB_PLL40_PAD consumes the pad and produces the fabric clock as its output.

```
SB_PLL40_PAD #(
    .FEEDBACK_PATH("SIMPLE"),
    .DIVR(4'd0), // DIVR = 0
    .DIVF(7'd73), // DIVF = 73
    .DIVQ(3'd5), // DIVQ = 5
    .FILTER_RANGE(3'b001)
) pll ( .PACKAGEPIN(clk_in), .PLLOUTCORE(pll_clk), .LOCK(pll_locked), ... );
```

The output frequency follows the iCE40 PLL equation:

$$\begin{aligned} F_{\text{out}} &= F_{\text{in}} * (DIVF + 1) / (2^{\text{DIVQ}} * (DIVR + 1)) \\ &= 12 \text{ MHz} * (73 + 1) / (2^5 * (0 + 1)) \\ &= 12 \text{ MHz} * 74 / 32 \\ &= 27.75 \text{ MHz} \end{aligned}$$

These dividers are not computed by hand. They come from the icestorm tool, `icepll -i 12 -o 27.75`, where `-i` is the input MHz and `-o` the desired output MHz.

There is also an optional divide-by-two stage gated by the `USE_CLK_DIVIDER` parameter. When it is set to 1, a toggle flip-flop halves `pll_clk` before the global buffer, so the PLL can run at a frequency it locks cleanly while the SoC sees half of it. Both paths drive `clk` through an `SB_GB` global buffer so the clock reaches the whole fabric. `resetn` is held until both the reset counter saturates and `pll_locked` is high, so the SoC only leaves reset once the PLL has locked.

```
generate
if (USE_CLK_DIVIDER) begin : gen_clk_divider
    reg clk_div = 1'b0;

    always @(posedge pll_clk) begin
        clk_div <= ~clk_div;
    end

    SB_GB clk_gb (
        .USER_SIGNAL_TO_GLOBAL_BUFFER (clk_div),
        .GLOBAL_BUFFER_OUTPUT (clk)
    );
end else begin : gen_no_clk_divider
    SB_GB clk_gb (
        .USER_SIGNAL_TO_GLOBAL_BUFFER (pll_clk),
        .GLOBAL_BUFFER_OUTPUT (clk)
    );
end
endgenerate

wire resetn = &reset_cnt && pll_locked; // leave reset only once the PLL locks
```

The divide-by-two stage was needed early in development. The achievable maximum frequency was then around 14 MHz, below the PLL floor of roughly 16 MHz, so the PLL could not be asked for the target clock directly. Running the PLL at about twice the target and halving it kept both the PLL and the fabric inside ranges they could meet. As the area and logic-depth reductions landed and the place-and-route seeds were swept, the achievable frequency rose clear of the floor, and the core was driven directly from the PLL. The divider survives only as an optional fall-back.

How to run it, and how to change the clock

The PLL is part of the board target, so a normal `make` build in `picosoc/` programs the board at 27.75 MHz. Changing the SoC clock is not a one-line edit, because the frequency leaks into firmware timing and into the place-and-route constraints. To move to a new frequency, update all of these so they agree:

1. **PLL dividers** in `picosoc/icebreaker.v`. Run `icepll -i 12 -o <MHz>` and paste the reported `DIVR`, `DIVF`, `DIVQ` and `FILTER_RANGE` into the `SB_PLL40_PAD` instance.
2. **USE_CLK_DIVIDER** in `picosoc/icebreaker.v`. Set it to 1 to run the SoC at half the PLL output, and halve the effective frequency used in the later steps.
3. **F_CPU** in `picosoc/blink.c`. This is the actual SoC clock in Hz. The UART baud divider is derived from it (`reg_uart_clkdiv = F_CPU / BAUD`), so a wrong value gives a garbled serial console.
4. **reg_uart_clkdiv** in `picosoc/benchmark.c`, which hard-codes the divider. Recompute it as `round(F_CPU / BAUD)` for the new clock.
5. **Timing constraints** in `picosoc/Makefile`. Update the `nextpnr-ice40 --freq <MHz>` target and the `icetime -c <MHz>` check so place-and-route optimises for, and verifies, the new clock.
6. **Confirm timing closes**. After `make`, check that the `icetime` or `nextpnr` report shows a maximum frequency at or above the target. If it fails, back the frequency off, enable `USE_CLK_DIVIDER`, run the seed sweep, or reduce the critical path by enabling the cache or the compact divider.

Results

Raising the clock from 12 to 27.75 MHz more than doubles it, and that gain multiplies straight through the cache's cycle-count savings into wall-clock speed. Static timing analysis with `icetime` estimated the design at around 18 to 19 MHz, but the open-source tools are conservative, and on the board the design ran reliably at 27.75 MHz, which is the clock used for the competition. That clock also made ours the highest-power entry, near 0.81 W against the lowest team's 0.66 W, which is the cost of the speed.